

MULTI-AGENT UAV CAPTURE AND EVASION WITH ADVERSARIAL REINFORCEMENT LEARNING

Rishikesh Narayana & Owen Murphy | SysLab 2026

The background is a light gray color. It is decorated with several overlapping squares of different shades of gray. Some squares are solid, while others have a thin white border. The squares are arranged in a way that creates a sense of depth and layering, with some appearing to be in front of others.

01

Background

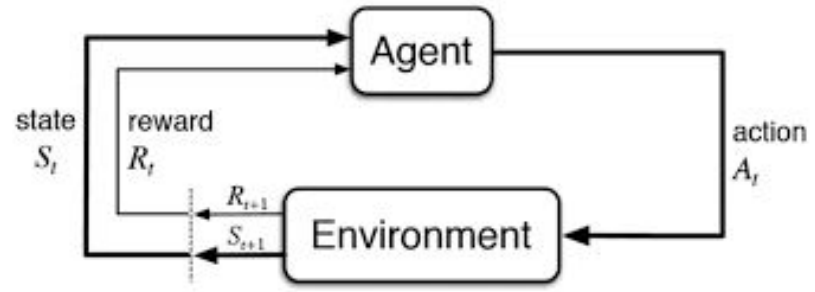
Background

- Existing machine learning algorithms for drones rely on high-fidelity sensors or global data.
- Autonomous systems lack adaptability in search-and-rescue (SAR), defense, and other rapidly changing situations.



Reinforcement Learning

- Agent: The learner or decision maker
- Environment: The world the agent interacts with
- Action (A): What the agent can do
- State (S): The situation the agent is in
- Reward (R): Feedback signal from the environment

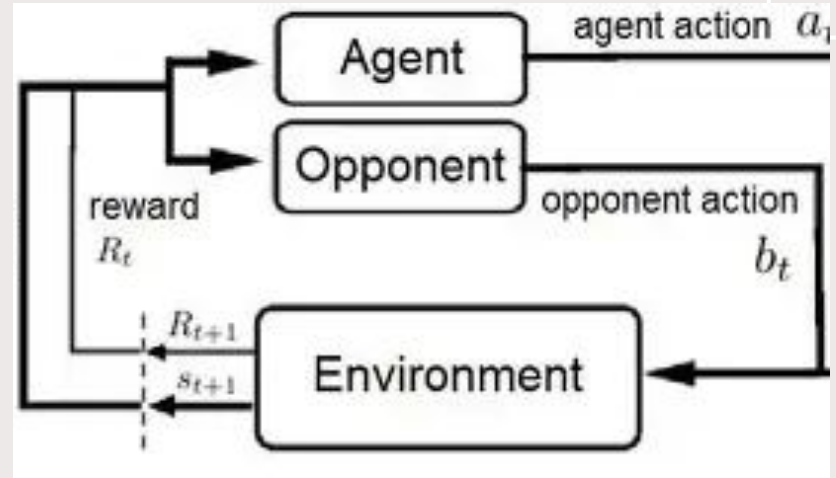


Types of Reinforcement Learning



Adversarial Reinforcement Learning

- Tool for teaching systems both offensive & defensive strategies (Baker et al.)
- Two (or more) agents influence each other through self-play



The background features a light gray surface with several overlapping squares of varying shades of gray and white. Some squares have rounded corners, while others are sharp. They are arranged in a way that creates a sense of depth and layering, with some squares appearing to float above others.

02

Objectives

Objectives

- Design and Implement adversarial reinforcement learning algorithms to 2 drones, one to capture and one to evade.
- First simulate in 3D python simulators, then transfer to real-world drones



Current Solutions

- MEADDQN (Double Deep Neural Network) achieves strong simulation results, but depends on global state information and is extremely slow
- OPEN MARL (Modular) show high capture rates in simulation, yet require expensive hardware setups and 100s of hours of training for new instance cases
- PSO-M3DDPG (Particle Swarm Optimization) focuses on simulation-based optimization with global info. Cannot transfer to real-world testing.

Novelty

- Low-cost small-drones with exclusive visual perception (bearing + distance via YOLOv5/AprilTags)
- Domain randomization for sim-to-real transfer



AprilTag Robots



Microsoft Airsim Domain Randomization

The background features a light gray surface with several overlapping squares of varying shades of gray and white. These squares are positioned primarily along the left and right edges, creating a layered, architectural effect. The central text is framed by a thin white square border.

03

**Simulation
Methodology**

Proximal Policy Optimization (PPO)

- “Traditional” RL can be unstable; PPO uses a clip function to limit policy deviation

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

Where:

- $r_t(\theta)$ = probability ratio of new vs. old policy
- $\hat{A}_t$ = advantage estimate (how good the action was compared to expected)
- ϵ = small threshold (e.g., 0.2)

- Unlike “Traditional” RL, PPO can reuse batches of experience, improving data efficiency and learning speed

PPO Model Comparison

Traditional PPO

- Uses a fixed clipping range to limit policy deviation, preventing destructive changes.

Adaptive Clip-rate

- Changes clipping range based on agent advantage at any given state

Kullback-Leibler Penalty

- Uses KL divergence penalty to control policy deviation

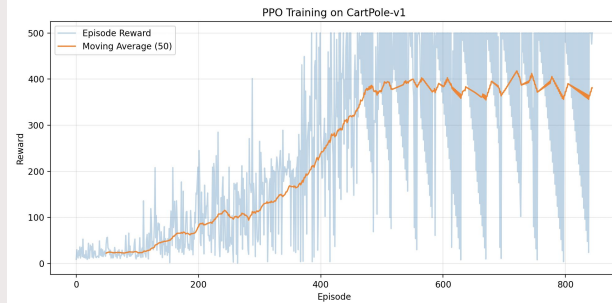
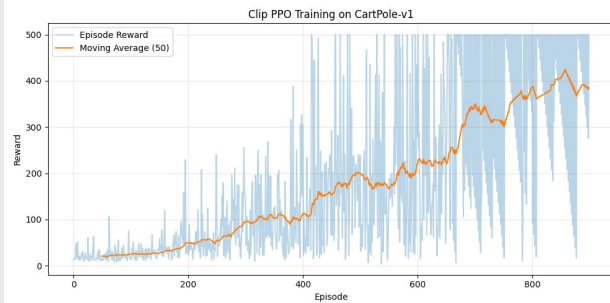
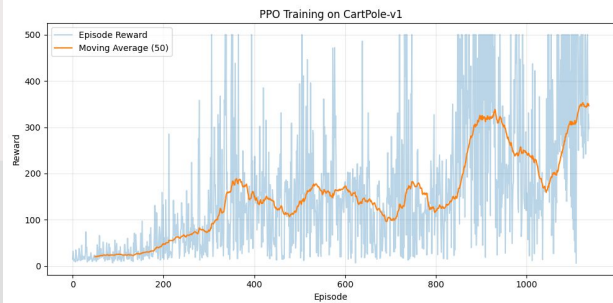
$$L^{clip}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{CLIP}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

$$A^t = Q^{\pi_{\theta}}(a_t|s_t) - V^{\pi_{\theta}}(s_t)$$

$$\epsilon = \frac{\alpha}{\sigma(A_t) + \delta} \text{ (Chen et al., 2018)}$$

$$L^{KL}(\theta) = \mathbb{E}_t[r_t(\theta)A_t] - \beta \mathbb{E}_t[D_{KL}(\pi_{old} || \pi_{new})] \text{ where}$$

$$D_{KL}(\pi_{old} || \pi_{new}) = \sum \pi_{old}(a|s) \cdot \log\left(\frac{\pi_{old}(a|s)}{\pi_{new}(a|s)}\right)$$



PPO Model Comparison

Traditional PPO

- Uses a fixed clipping range to limit policy deviation, preventing destructive changes.

Adaptive Clip-rate

- Changes clipping range based on agent advantage at any given state

Kullback-Leibler Penalty

- Uses KL divergence penalty to control policy deviation

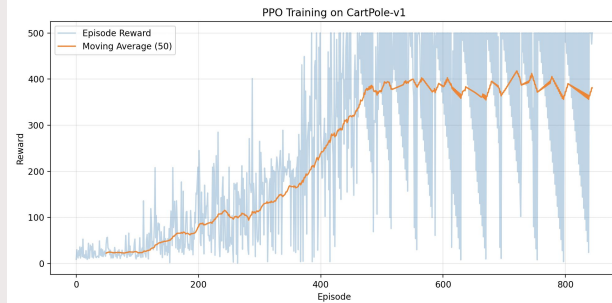
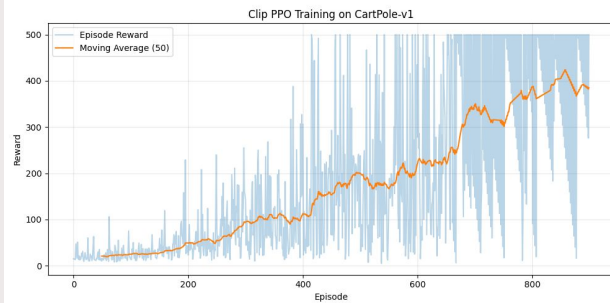
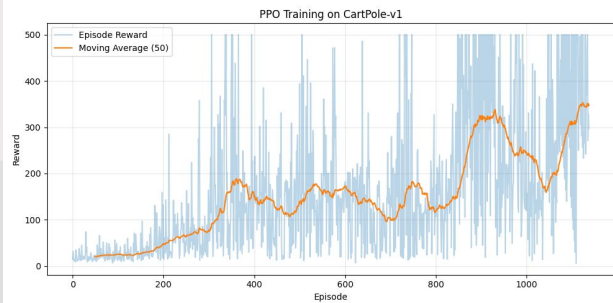
$$L^{clip}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{CLIP}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

$$A^t = Q^{\pi_{\theta}}(a_t|s_t) - V^{\pi_{\theta}}(s_t)$$

$$\epsilon = \frac{\alpha}{\sigma(A_t) + \delta} \text{ (Chen et al., 2018)}$$

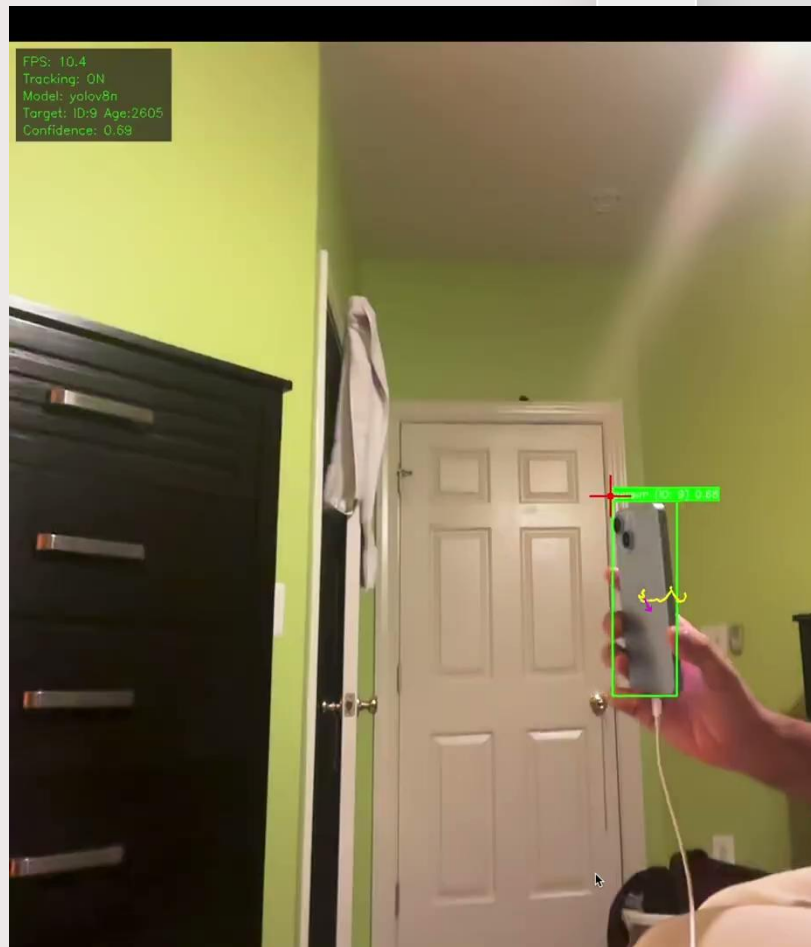
$$L^{KL}(\theta) = \mathbb{E}_t[r_t(\theta)A_t] - \beta \mathbb{E}_t[D_{KL}(\pi_{old} || \pi_{new})] \text{ where}$$

$$D_{KL}(\pi_{old} || \pi_{new}) = \sum \pi_{old}(a|s) \cdot \log\left(\frac{\pi_{old}(a|s)}{\pi_{new}(a|s)}\right)$$

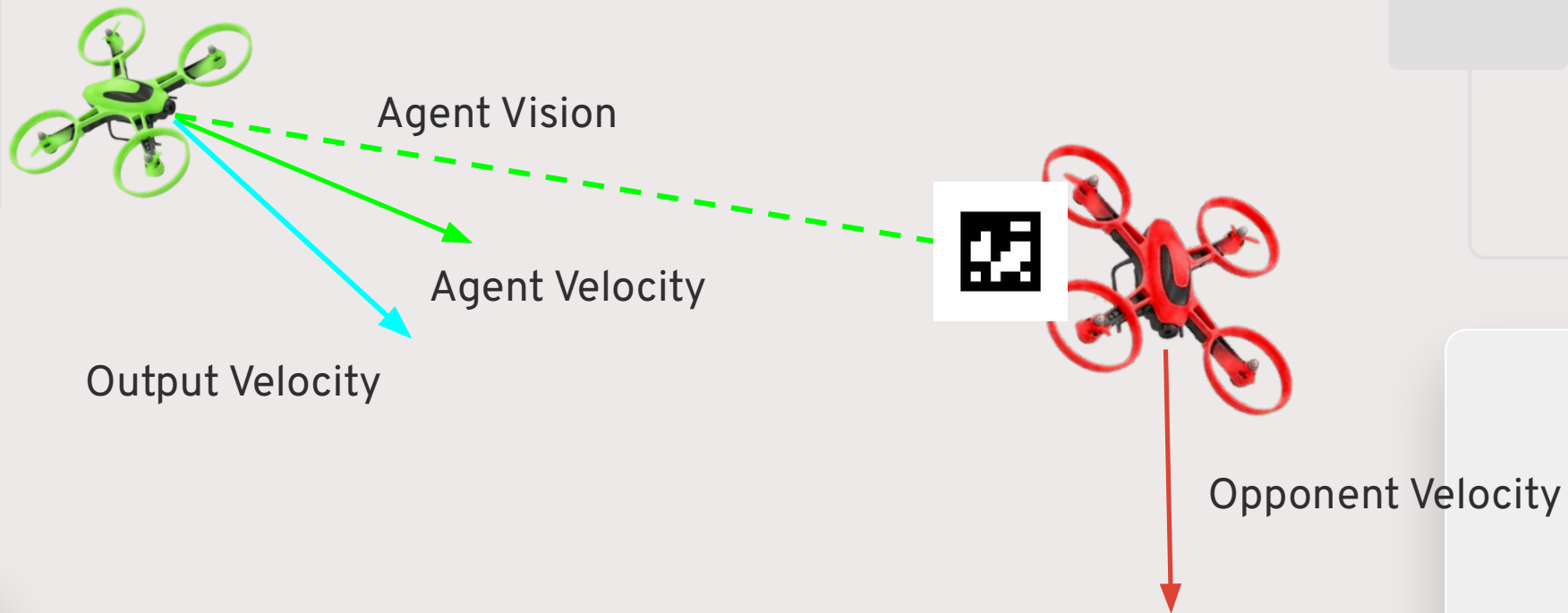


PPO Inputs

- Agent velocity vector: speed and direction of agent
- Opponent velocity vector and relative angle: captures speed and direction of opponent. Calculate relative angle via position in FOV.
- Opponent relative position vector to agent.

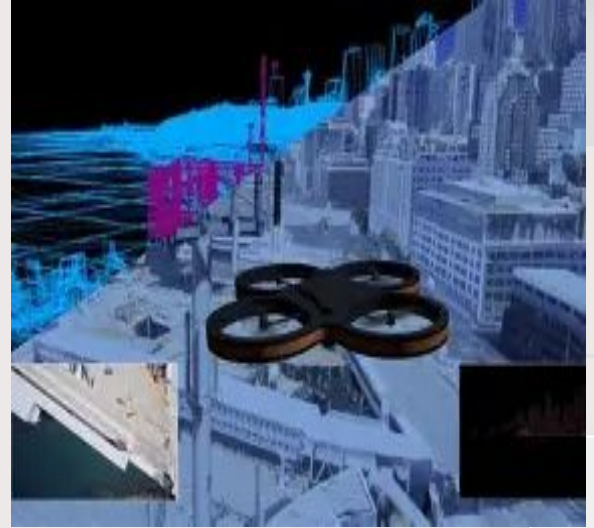


PPO Inputs Visual



Microsoft Airsim

- Simulation platform developed by Microsoft for UAVs
- Allows for safe simulating & prototyping with domain randomization, varying conditions to make policies transferable to real drone
- Connect with MavLink and use Python API for simulation and controlling drone movement



AirSim Training

- stable_baselines3 PPO model
- Custom gymnasium environment class for Tracker & Evader
- At each step, calculates own position $\langle x, y, z \rangle$, velocity $\langle vx, vy, vz \rangle$, and creates a relative distance vector between itself and opponent
- Outputs a new velocity vector
- Communicates with AirSim

Model initialization

```
model = PPO(  
    "MlpPolicy",  
    env,  
    learning_rate=5e-4,  
    n_steps=512,  
    batch_size=128,  
    n_epochs=10,  
    gamma=0.98,  
    gae_lambda=0.95,  
    clip_range=0.2,  
    ent_coef=0.05,  
    vf_coef=0.5,  
    max_grad_norm=0.5,  
    tensorboard_log="./logs/",  
    policy_kwargs=dict(net_arch=[128, 128]),  
    verbose=1,  
    device='cuda' if torch.cuda.is_available() else 'cpu'  
)
```

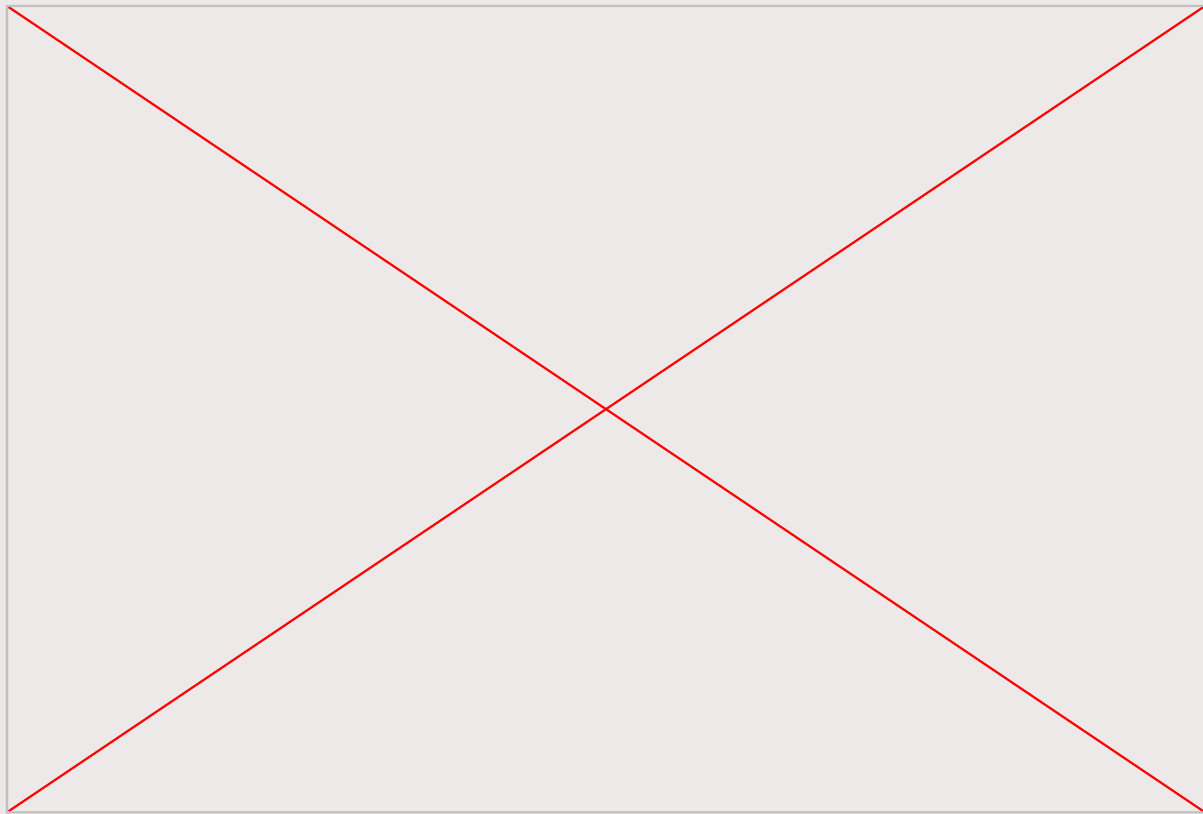
```
self.client.moveByVelocityAsync(  
    vx, vy, vz,  
    duration=self.dt,  
    vehicle_name=self.tracker_name  
)
```

Action

Input

```
rel_x = gx - tx  
rel_y = gy - ty  
rel_z = gz - tz  
  
obs = np.array([  
    rel_x,  
    rel_y,  
    rel_z,  
    tvx,  
    tvy ,  
    tvz  
  
], dtype=np.float32)  
  
return obs
```

AirSim PPO Demo #1



Evader Reward Function

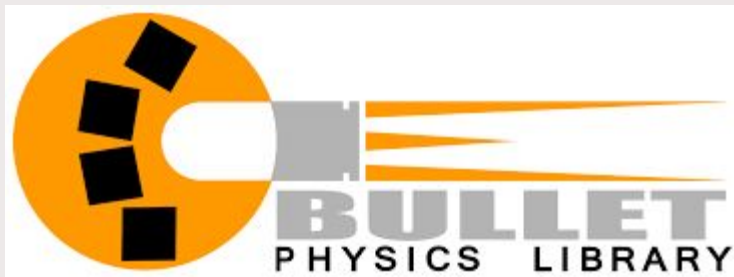
- Evader rewarded for:
 - Far distance to Tracker (distance_reward)
 - Achieving a higher distance compared to last action (progress_reward)
 - Avoiding the same direction & velo for long time (degenerate reward)
 - Added smoothness control to both
- Penalized for:
 - Flying out of bounds
 - Getting caught ($d < 2m$)

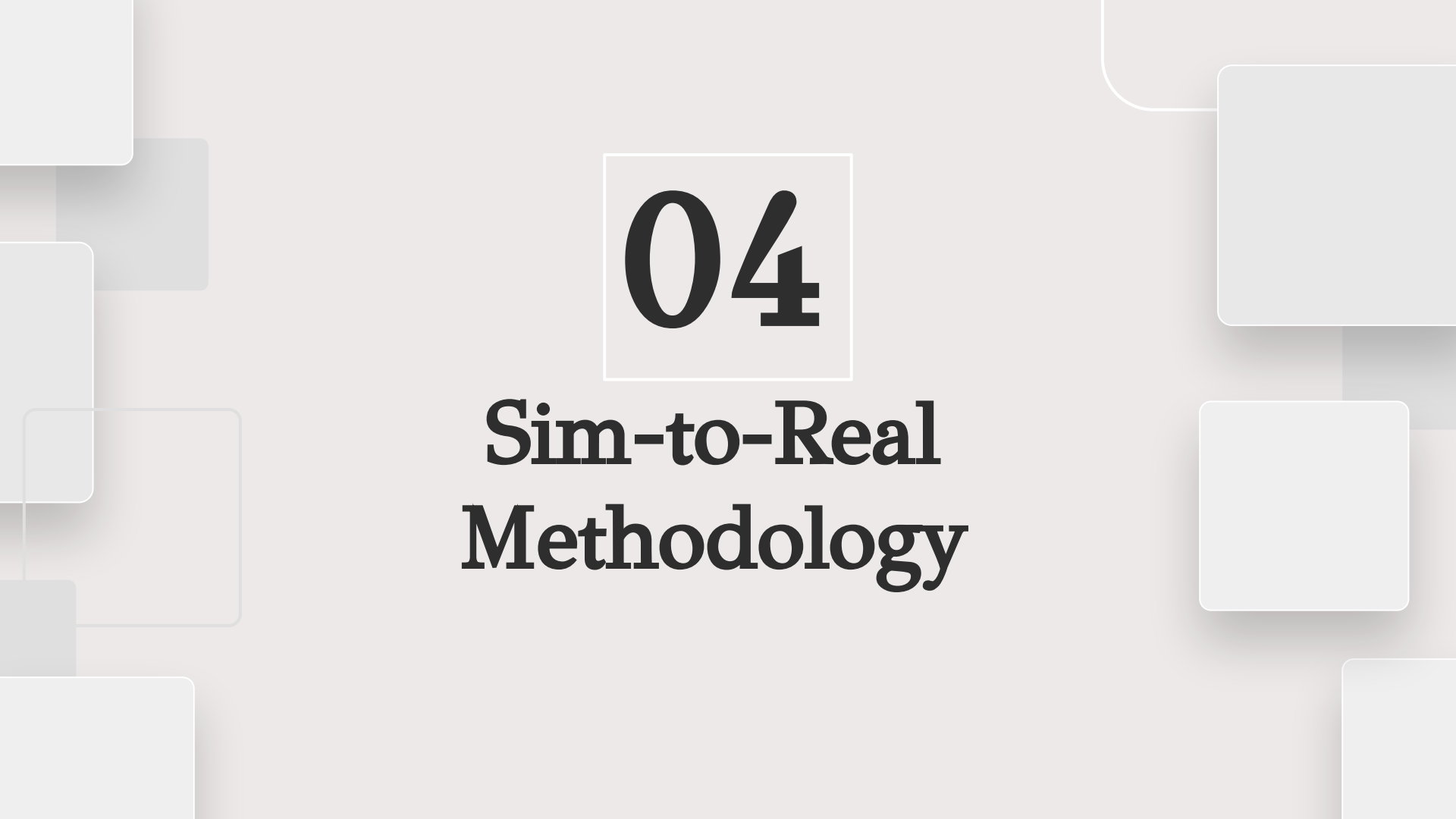
Tracker Reward Function

- Tracker rewarded for:
 - Close distance to Evader
(distance_reward) using tanh
 - Achieving a lower distance compared to last action (progress_reward)
 - Staying within a certain range of distance (zone_bonus)
 - Keeping Evader in center of FOV
- Penalized for:
 - Flying out of bounds
 - Collisions / distance $\leq 0.5\text{m}$

Pivot to PyBullet

- AirSim provides high-fidelity physics and sensor simulation, but it's computationally heavy and significantly slows down training and iteration due to rendering and simulation overhead.
- To improve efficiency, we offloaded the environment to PyBullet, which offers a lighter-weight physics engine that enables faster simulation cycles



The background features a light gray surface with several overlapping squares of varying shades of gray. Some squares have rounded corners, while others are sharp. They are arranged in a way that creates a sense of depth and layering, with some squares appearing to be in front of others.

04

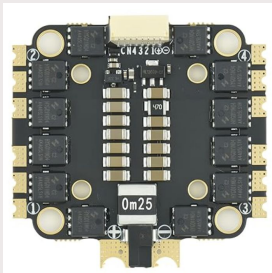
Sim-to-Real Methodology

Drone Hardware

Battery



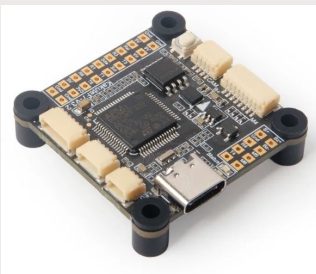
4-in-1 ESC



Brushless Motors



NVIDIA Jetson Nano



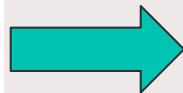
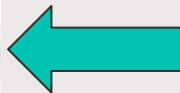
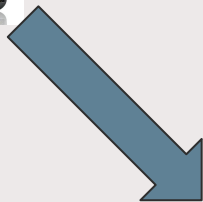
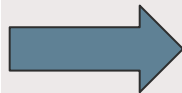
Flight Controller



2.4 GHz ELRS

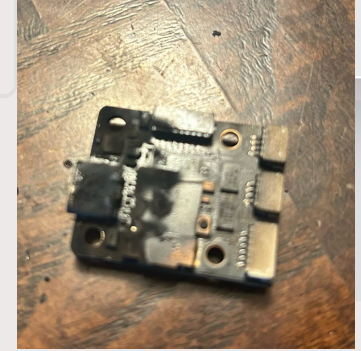
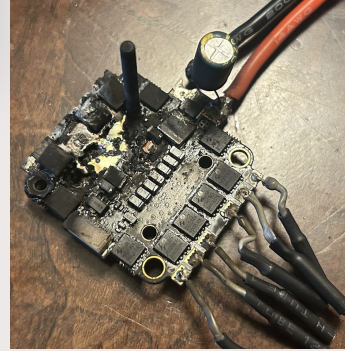


Radio Controller



Pivot to DJI Tello

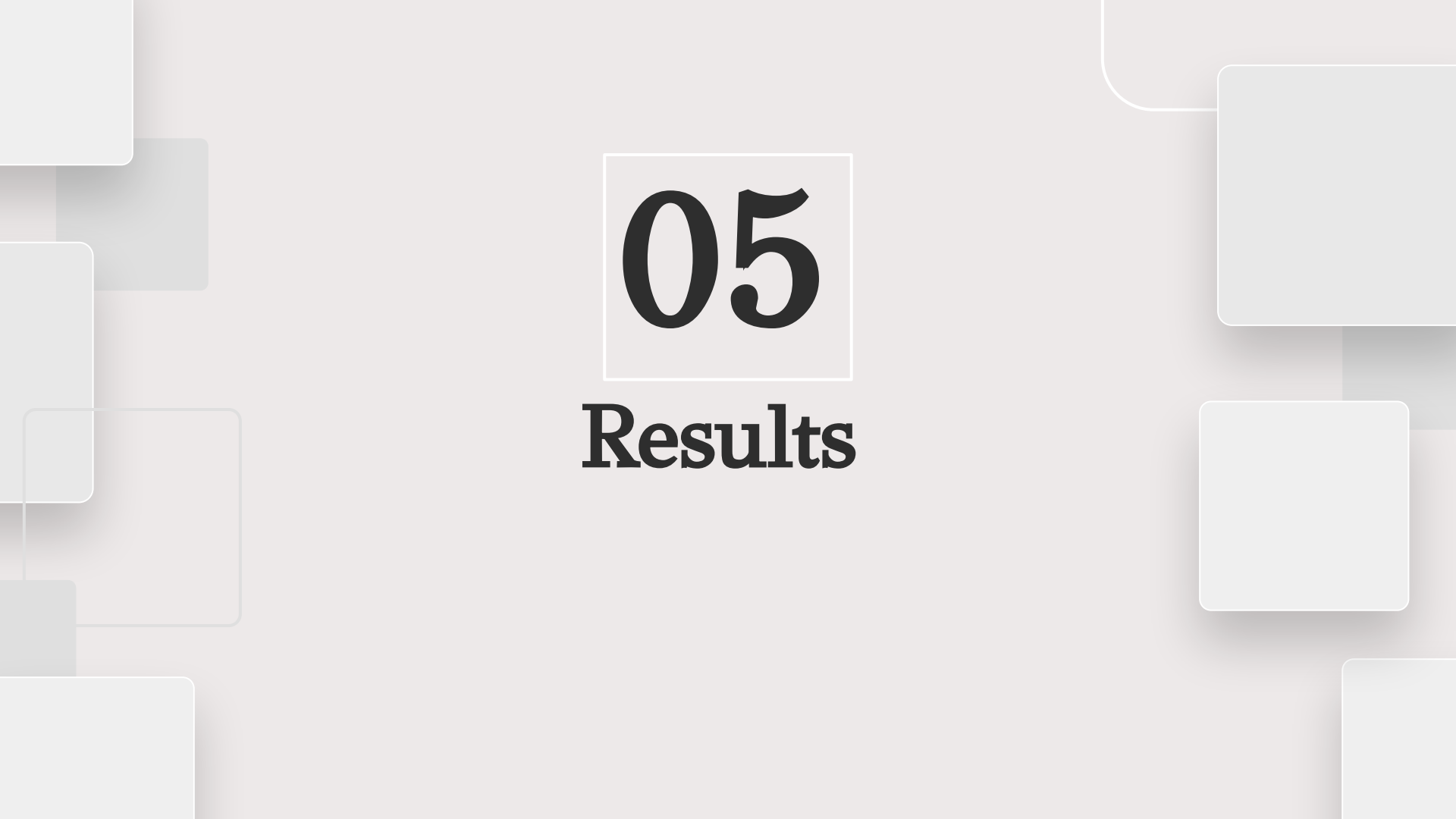
- Encountered large short circuit, resulting in burned and unusable flight controller and ESC.
- Due to difficulties in assembly, we switched to DJI Tellos which have a built in Python API for low level commands



DJI Tello EDU Drone

Testing Method

- For simplicity and lack of global positioning for evader, we've done real-world testing only for the tracker.
- Using April Tags hung from evader, we effectively have positioning data when the evader is seen by the tracker.

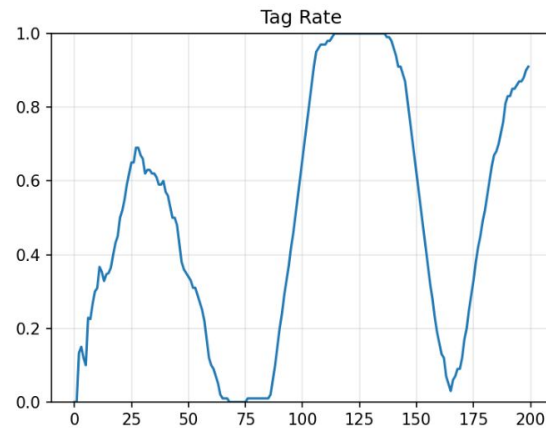
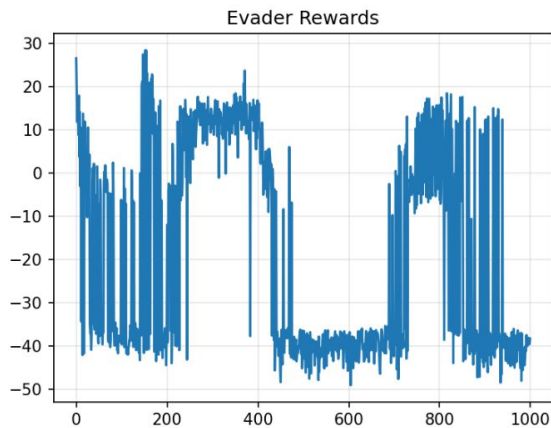
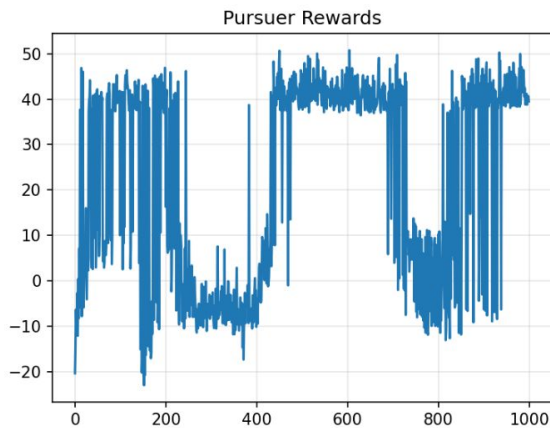
The background features several overlapping squares in various shades of light gray and white, some with thin white borders, creating a layered, geometric effect.

05

Results

Simulation Results

- Trained for 1000 episodes. 3000 timesteps per episode. 3,000,000 total timesteps
- Final Average Win Rate: 95% Tag Rate
- Oscillating reward graphs between training sessions. Graph could potentially converge towards either Pursuer/Evader



Real-World Results

- The pursuer demonstrated stable visual tracking of the AprilTag-equipped drone throughout dynamic motion sequences, as long as the April Tag was within pursuer FOV
- Successfully translated basic simulated pursuit behaviors into real-world control performance, enabling the pursuer to react to target movement.
- Experimental trials showed reliable target reacquisition after brief occlusions or rapid motion changes